UQAC
Département d'informatique
et de mathématique
Université du Québec à Chicoutimi

8INF878 – Intelligence artificielle agentique : Travail No. 2

AUSSI V3 – Jeux adversariaux et généralisation

Etudiants :
CAPELO DO NOVO Thomas
LOUET Noah
FOUGERAY Mattéo
BERTUCAT Antoine

13 novembre 2026

Table des matières

1. Introduction 2
2. Architecture des moteurs 2
   2.1. Minimax 2
   2.2. Monte Carlo Tree Search 3
3. Tic Tac Toe 4
   3.1. Modélisation 4
   3.2. Comparaison des algorithmes 4
4. Connect Four 5
   4.1. Modélisation 5
   4.2. Comparaison des algorithmes 6
5. Conclusion 7

1. Introduction

Ce travail est réalisé dans le cadre du cours 8INF878 – Intelligence artificielle agentique. Son objectif est de concevoir AUSSI V3 (*Agent Universel de Support Super Intelligent*), qui fait suite à AUSSI V2, fruit du Travail No. 1.

Désormais, AUSSI doit être capable de jouer à des jeux à deux joueurs et à tours alternés. AUSSI aura ici le choix entre le Tic Tac Toe et le Puissance 4 (Connect 4), chacun jouable en utilisant un moteur basé sur le Minimax ou sur la méthode MCTS (Monte Carlo Tree Search).

Ce rapport se basera plus sur ces différents jeux et les moteurs utilisés plutôt que sur l'architecture générale et l'organisation du code de notre projet. Pour retrouver ces informations, vous pouvez vous referrer au rapport du Travail No. 1.

2. Architecture des moteurs

Les deux moteurs sont écrits de façon générique : ils ne connaissent aucun jeu en particulier et communiquent uniquement avec l'interface abstraite `Game`. Chaque moteur est un agent dont la méthode `get_move(game, state)` retourne l'action choisie. Pour ajouter un jeu, il suffit d'écrire une sous-classe de `Game`, sans modifier les algorithmes de décision.

Tous les jeux héritent de la classe abstraite `Game`, qui définit le contrat commun utilisé par les moteurs. Elle impose de définir :

- `initial_state()` : l'état initial du jeu ;
- `player_to_move(s)` : le joueur qui doit jouer dans l'état `s` ;
- `actions(s)` : la liste des actions légales depuis l'état `s` ;
- `result(s, a)` : l'état obtenu en appliquant l'action `a` ;
- `is_terminal(s)` : indique si l'état `s` correspond à une fin de partie ;
- `winner(s)` : retourne le joueur gagnant, ou `None` en l'absence de gagnant ;
- `utility(s, player)` : retourne l'utilité exacte d'un état terminal du point de vue du joueur donné : `+1` pour une victoire, `-1` pour une défaite et `0` pour une partie nulle.

La classe fournit également `is_legal(s, a)` pour vérifier la légalité d'une action, `evaluate(s, player)` pour estimer la qualité d'un état non terminal et `display(s)` pour afficher un état dans la console.

Un état représente la configuration complète du jeu à un instant donné. Il doit être hashable afin de pouvoir être utilisé dans des structures de données adaptées à la recherche. De plus, `result(s, a)` doit produire un nouvel état sans modifier celui reçu en paramètre. Ainsi, les moteurs peuvent explorer différentes suites de coups sans altérer l'état réel de la partie.

Il est important de distinguer l'utilité exacte d'un état terminal de l'évaluation heuristique d'un état non terminal. L'utilité permet de reconnaître une victoire, une défaite ou une partie nulle. L'évaluation heuristique estime plutôt la qualité d'une position lorsque la recherche ne peut pas aller jusqu'à la fin de la partie. Cette séparation permet notamment à Minimax de prendre en compte les résultats réels lorsqu'ils sont connus, tout en utilisant une estimation lorsque la profondeur maximale est atteinte.

2.1. Minimax

Le moteur Minimax (`MinimaxAgent`) explore l'arbre de jeu en supposant que les deux joueurs jouent de manière optimale : le joueur courant cherche à maximiser l'évaluation de l'état, son adversaire à la minimiser. Trois mécanismes le rendent utilisable sur des jeux dont l'arbre est trop grand pour être exploré entièrement.

Élagage alpha-bêta. On propage deux bornes : α, la meilleure valeur déjà garantie pour MAX, et β, la meilleure valeur déjà garantie pour MIN. Dès qu'un nœud MAX obtient une valeur $v \geq \beta$, ou qu'un nœud MIN obtient $v \leq \alpha$, les actions restantes sont ignorées, car, en

théorie, l'adversaire n'autoriserait jamais d'atteindre ce nœud. Le résultat est identique à celui du Minimax classique, mais plus efficace.

Fonction d'évaluation. L'exploration s'arrête quand l'état est terminal ou quand la profondeur restante atteint 0. L'état est alors évalué par `eval_fn(état, joueur)`, par défaut la méthode `evaluate` du jeu. Cette heuristique est toujours calculée du point de vue du joueur à la racine, que l'on maximise aux nœuds MAX et que l'on minimise aux nœuds MIN. Chaque jeu fournit la sienne (voir les sections suivantes).

Approfondissement itératif (IDS) et limite de temps. Le moteur lance successivement des recherches de profondeur 1, 2, …, `max_depth`. Une limite de temps (`time_limit`, 2 s par défaut) est vérifiée à chaque nœud : si elle est dépassée, une exception interrompt la recherche en cours et on conserve le meilleur coup de la dernière profondeur terminée. L'agent répond donc toujours dans le temps imparti. Le meilleur coup de la profondeur $d-1$ est de plus essayé en premier à la profondeur d, ce qui améliore l'élagage, puisque alpha-bêta est d'autant plus efficace que les bons coups sont examinés tôt. Les jeux peuvent aussi fournir leurs actions dans un ordre favorable : le Puissance 4 renvoie les colonnes du centre vers les bords.

L'agent expose sur l'interface la dernière profondeur complétée, le nombre de nœuds explorés et le temps de recherche.

2.2. Monte Carlo Tree Search

Le moteur MCTS n'utilise aucune heuristique : il estime la valeur d'un coup à partir de parties jouées au hasard jusqu'à leur terme. Il construit progressivement un arbre dont chaque nœud mémorise son nombre de visites n et la somme W des récompenses obtenues. Tant que le temps imparti (`time_limit`) n'est pas écoulé, il répète quatre phases :

1. **Sélection** : depuis la racine, on descend dans l'arbre tant que le nœud courant est entièrement développé et non terminal, en choisissant l'enfant qui maximise la borne UCT :

$$\mathrm{UCT} = \frac{W}{n} + C\sqrt{\ln \frac{N}{n}}$$

   où N est le nombre de visites du parent et C la constante d'exploration (1 par défaut). Le premier terme favorise les coups qui ont bien réussi (exploitation), le second ceux qui ont été peu essayés (exploration). Aux nœuds où c'est l'adversaire qui joue, la moyenne est inversée, car il choisit le pire coup pour l'agent.
2. **Expansion** : on tire au hasard une action non encore essayée du nœud sélectionné et on crée l'enfant correspondant.
3. **Simulation** : à partir du nouvel état, les deux joueurs jouent des coups aléatoires jusqu'à une position terminale. La récompense vaut $+1$ si le joueur à la racine gagne, -1 s'il perd et 0 pour un match nul.
4. **Rétropropagation** : la récompense remonte jusqu'à la racine, en incrémentant les visites et en cumulant la valeur de chaque nœud traversé.

À la fin du temps imparti, le coup joué est celui de l'enfant de la racine le plus visité (la valeur moyenne départage les égalités), ce qui est plus robuste que de prendre la meilleure moyenne. L'agent expose le nombre d'itérations réalisées et le temps de recherche.

Contrairement au Minimax, MCTS dépend du nombre de simulations réalisables dans le temps imparti.

3. Tic Tac Toe

3.1. Modélisation

Le Tic Tac Toe se joue sur une grille de 3×3 cases où deux joueurs posent chacun leur tour un jeton dans une case vide, e premier qui aligne trois jetons gagne. Voici la modélisation que nous avons mis en place :

- **État** : un tuple de 3 tuples d'entiers (0 pour une case vide, 1 ou 2 pour les jetons des joueurs).
- **Joueur courant** : il est déduit de l'état en comptant les jetons : si les deux joueurs en ont posé autant, c'est le joueur 1 qui joue, sinon c'est le joueur 2.
- **Actions** : le couple `(ligne, colonne)` de chaque case vide.
- **Résultat** : une copie de la grille dans laquelle on rajoute le jeton joué par le joueur.
- **Test de fin** : on vérifie les 8 lignes possible et on retourne le gagnant s'il existe, un match nul si la grille est pleine, sinon la partie continue.

Fonction d'évaluation. Elle est calculée du point de vue d'un joueur J et somme deux composantes. Les jetons de l'adversaire comptent négativement avec un facteur 1,01 : l'évaluation est ainsi légèrement pessimiste, ce qui évite d'attribuer la même valeur à des positions que l'on souhaite départager.

1. **Composante positionnelle** : chaque jeton rapporte le poids de sa case, 4 pour le centre, 3 pour les coins et 2 pour les côtés, ce qui reflète le nombre de lignes gagnantes passant par chaque case.
2. **Composante par ligne** : pour chacune des 8 lignes, si elle ne contient que des jetons de J (ou que des jetons adverses), on ajoute (ou retire) un poids selon le nombre de jetons : 10 pour un, 100 pour deux et 100 000 pour trois, c'est-à-dire une victoire. Une ligne contenant des jetons des deux joueurs ne peut plus être gagnée et est ignorée.

Le grand poids d'un alignement complet garantit que, lorsque la recherche atteint une position gagnante, celle-ci domine.

3.2. Comparaison des algorithmes

Il y a au plus $9! = 362880$ séquences de coups au Tic Tac Toe, l'arbre de jeu est donc assez petit. Minimax avec une profondeur de 9 le parcourt entièrement (en moyenne 14 000 nœuds par coup face à un joueur aléatoire) et ne perd jamais. Pour comparer les moteurs, nous avons fait jouer 10 parties par confrontation, en alternant le joueur qui commence. Les paramètres sont : Minimax avec une profondeur maximale de 9 (ou de 2 pour la recherche courte) et 2 s par coup, et MCTS avec 1 s par coup, soit environ 20 000 itérations par coup.

Confrontation	Parties	Victoires du 1er algorithme	Victoires du 2e algorithme	Nuls
Minimax (prof. 9) contre aléatoire	10	10	0	0
MCTS (1 s) contre aléatoire	10	10	0	0
Minimax (prof. 9) contre Minimax (prof. 9)	10	0	0	10
Minimax (prof. 9) contre MCTS (1 s)	10	0	0	10
Minimax (prof. 2) contre MCTS (1 s)	10	0	0	10
MCTS (1 s) contre MCTS (1 s)	10	0	0	10

Tableau 1 – Résultats des confrontations au Tic Tac Toe

On observe que :

- Les deux moteurs battent largement un joueur aléatoire.
- Les confrontations Minimax/Minimax, Minimax/MCTS et MCTS/MCTS donnent toujours des égalités, néanmoins dans le cas Minimax/MCTS, Minimax pourrait gagner si MCTS avait un temps de reflexion plus petit. Car les évaluations de MCTS sont basées sur le hasard.
- Même une recherche Minimax très courte (profondeur 2) suffit à égaliser face à MCTS, grâce à l'évaluation qui valorise le centre et les alignements.

Les résultats sont issus d'un petit nombre de parties et sont donc indicatifs. Le Tic Tac Toe est trop simple pour départager les moteurs.

4. Connect Four

4.1. Modélisation

Le Puissance 4 se joue sur une grille de 6 lignes et 7 colonnes. Les joueurs déposent à tour de rôle un jeton dans une colonne, le jeton tombe à la case libre la plus basse. Le premier qui aligne quatre jetons (horizontalement, verticalement ou en diagonale) gagne (si la grille est pleine, la partie est nulle). Le jeu est bien plus grand que le Tic Tac Toe : l'arbre de jeu compte plusieurs milliers de milliards de positions, et aucun moteur ne peut l'explorer en entier dans un temps raisonnable.

- **État** : un tuple de 6 tuples de 7 entiers (0 pour vide, 1 ou 2 pour les joueurs), la case $(0, 0)$ étant en haut à gauche.
- **Joueur courant** : déduit du nombre de jetons posés, le joueur 1 commence.
- **Actions** : l'indice des colonnes dont la case du haut est vide. Elles sont retournées dans l'ordre 3, 4, 2, 5, 1, 6, 0, c'est-à-dire du centre vers les bords. Les coups centraux sont en général meilleurs car ils participent à plus d'alignements possibles ; les examiner en premier augmente le nombre de coupes de l'élagage alpha-bêta.
- **Résultat** : la gravité est simulée en cherchant, depuis le bas, la première case vide de la colonne choisie.
- **Test de fin** : on cherche un alignement de quatre jetons dans les quatre directions ; la partie est nulle si la ligne du haut est entièrement remplie.

Fonction d'évaluation. Comme au Tic Tac Toe, elle est calculée du point de vue d'un joueur J, et les termes adverses sont multipliés par 1,01. Elle somme deux composantes :

1. **Fenêtres de quatre cases** (horizontales et diagonales) : toutes les fenêtres sont générées une seule fois. Une fenêtre contenant des jetons des deux joueurs est ignorée. Sinon elle rapporte un poids selon le nombre de jetons du même joueur : 0 pour un, 100 pour deux, 1 000 pour trois et 100 000 pour quatre (victoire). Le poids est ajouté si la fenêtre appartient à J et retiré si elle appartient à l'adversaire.
2. **Colonnes (alignements verticaux)** : pour ch aque colonne, on regarde la pile de jetons de même couleur au sommet (n jetons) et le nombre de cases libres au-dessus. Si n plus les cases libres permet encore d'atteindre quatre, la colonne a un potentiel et rapporte le poids correspondant à n. Une pile d'un seul jeton rapporte un poids lié à sa colonne, `[2, 6, 9, 16, 9, 6, 2]`, qui favorise le centre.

4.2. Comparaison des algorithmes

Le Puissance 4 est le jeu où les différences entre moteurs apparaissent. Voici les confrontations réalisées, en alternant le joueur qui commence. Les parties de Minimax sont limitées à 0,5 s par coup (profondeur maximale 7) et celles de MCTS à 0,3 s ou 0,5 s par coup.

Confrontation	Parties	Victoires du 1er moteur	Victoires du 2e moteur	Nuls
Minimax (prof. 7, 0,5 s) contre aléatoire	8	8	0	0
MCTS (0,3 s) contre aléatoire	8	8	0	0
Minimax (prof. 7, 0,5 s) contre MCTS (0,5 s)	8	8	0	0
Minimax (prof. 3, 0,5 s) contre MCTS (0,5 s)	6	5	1	0
Minimax (prof. 5) contre Minimax (prof. 2)	4	2	2	0

Tableau 2 – Résultats des confrontations au Puissance 4

Les statistiques d'exploration montrent la limite des moteurs :

- Avec 0,5 s par coup, Minimax termine en moyenne la profondeur 5,4 sur une profondeur maximale demandée de 7, en explorant environ 3 800 nœuds. L'approfondissement itératif garantit une réponse dans le temps imparti, mais la profondeur atteinte dépend de la vitesse de la machine.
- MCTS réalise environ 500 itérations en 0,3 s, ce qui est peu pour un arbre aussi large, et ses simulations sont faites avec des coups totalement aléatoires.

On en retient que :

- Les deux moteurs battent facilement un joueur aléatoire.
- Face à MCTS, Minimax avec une profondeur suffisante l'emporte à chaque partie. L'évaluation heuristique (centre, alignements) lui donne une connaissance du jeu que MCTS ne peut pas reconstituer avec si peu de simulations. Même limité à une profondeur 3, Minimax gagne 5 parties sur 6, ce qui montre que la qualité de la fonction d'évaluation compte autant que la profondeur.
- Le duel Minimax (profondeur 5) contre Minimax (profondeur 2) est partagé : les deux moteurs étant déterministes, chacun gagne lorsqu'il joue dans une certaine configuration. Une

plus grande profondeur n'assure donc pas toujours la victoire, surtout lorsque l'heuristique n'est pas parfaite.

Comme pour le Tic Tac Toe, ces résultats reposent sur un nombre limité de parties et doivent être interprétés comme des tendances. Avec davantage de temps de calcul (ou une implémentation plus rapide), MCTS profiterait davantage du nombre de simulations, alors que Minimax gagnerait en profondeur.

5. Conclusion

Ce travail nous a permis de faire évoluer AUSSI vers une troisième version capable de jouer à des jeux à deux joueurs et à tours alternés. Les moteurs Minimax et MCTS ont été écrits de façon générique, ce qui permet de les utiliser sur le Tic Tac Toe et le Puissance 4 sans modifier leur fonctionnement. L'ajout d'un nouveau jeu repose ainsi sur son implémentation de l'interface Game.

Les résultats montrent que les deux algorithmes sont efficaces contre un joueur aléatoire. Mais la comparaison dépend du jeu et des paramètres utilisés. Au TicTacToe les deux moteurs arrivent toujours à égalité alors qu'au Connect 4 les différences sont plus visibles. Ce dernier nous a permis de montrer que lorsque Minimax a une profondeur de recherche suffisante il bat MCTS. MCTS présente de son côté l'avantage de pouvoir améliorer ses estimations en réalisant davantage de simulations, mais celles-ci reposent sur des coups aléatoires et ne lui permettent pas toujours de trouver les meilleures positions.

Enfin ces résultats ne sont qu'indicatifs, il faudrait faire plus de parties pour approdondir la comparaison et voir jusqu'où chaque algorithme pourrait améliorer ses performances.